DataWeave

Link: https://www.curietech.ai/mulesoft-dataweave

AI link: https://app.curietech.ai/test-drive

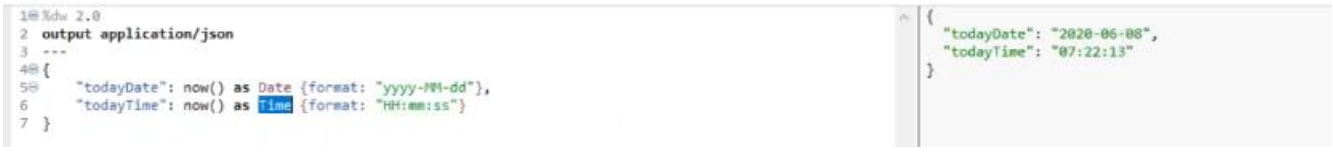
DataType conversion:

Use default “”/ or null. So, in case of converting null, default value is useful.

```
%dw 2.0
output application/java
var employee = {
    empName: "Michael Silva",
    empLocation: null,
    empExperience: null,
    isActive: "true"
}
---
{
    empName: employee.empName as String default "",
    empLocation: employee.empLocation as String default "",
    empExperience: employee.empExperience as Number default 0,
    isActive: employee.isActive as Boolean default false
}
```

Date & Time:

For a specific format



LocalDateTime: it is the combination of date and time

DateTime: it is the combination of datetime and timezone

```
%dw 2.0
output application/json
---
{
    "todayDate": now() as Date {format: "yyyy-MMM-dd"},
    "todayTime": now() as Time {format: "HH:mm:ss.SSS"},
    "currentHour": now() as Time {format: "HH"},
    "currentDateTime": now() as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss.SSS"},
    "UTCTime": ((now() >> "UTC") >> "IST") as DateTime {format: "yyyy-MM-dd HH-mm-ss"},
    "ISTTime": now() >> "IST",
    "ESTTime": now() >> "EST"
}
```

Output:

Example:

```
%dw 2.0
output application/java
---
{
    emp_id: payload.empID as Number,
    emp_name: payload.empName,
    emp_address: payload.empAddress,
    emp_status: payload.empStatus,
    emp_join_date: payload.empJoinDate as LocalDateTime {format: "yyyy-MM-dd'T'HH:mm:ss'Z'"} as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}
}
```

First LocalDateTime is we are getting in that format, we need to change it into required format using second LocalDateTime.

>> Generally timezone is in UTC format.

UTC to EST coversion:

```
%dw 2.0
output application/java
---
{
    emp_id: payload.empID as Number,
    emp_name: payload.empName,
    emp_address: payload.empAddress,
    emp_status: payload.empStatus,
    emp_join_date: (payload.empJoinDate >> "EST") as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}
}
```

>> if there is no input of dateTime use default.

```
%dw 2.0
output application/java
---
{
    emp_id: payload.empID as Number,
    emp_name: payload.empName,
    emp_address: payload.empAddress,
    emp_status: payload.empStatus,
    emp_join_date: (payload.empJoinDate >> "EST")  as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"} default null
}
```

Flow control (or) conditional based statements

1. If
2. if else
3. if else ladder

DataWeave functions

1. User-defined functions
2. Pre-defined functions
   - Date functions
     1. now
     2. daysBetween
     3. isLeapYear
     4. Date decomposition

If else:

There are 2 ways: 1. object level (full)
2. field level

1. Object level:

```
%dw 2.0
output application/json
var country = "USA"
var city = "New York"
---
if(country == "USA") {
    country: "USD",
    city: city
} else {
    country: "None",
    city: "None"
}
```

```
{
  "country": "USD",
  "city": "New York"
}
```

```
%dw 2.0
output application/json
var country = "India"
var city = "Delhi"
---
if(country == "USA") {
    currency: "USD",
    city: city
} else if(country == "India"){
    currency: "IRS",
    city: "Delhi"
} else {
    currency: "None",
    city: "None"
}
```

2. Field Level

```
%dw 2.0
output application/json
var country = "India"
var city = null
---
{
    currency: if(country == "USA") "USD" else if(country == "India") "IRS" else "None",
    city: if(city != null) city as String else ""
}
```

```
{
  "currency": "IRS",
  "city": ""
}
```

==========

Functions:

DataWeave (used in MuleSoft) provides a rich set of functions across several modules. Below is a categorized list of commonly used functions in DataWeave 2.0.

Core Functions

* now(), today(), time()
* upper(), lower(), trim(), replace(), substring()
* sizeOf(), isEmpty(), isBlank()
* round(), floor(), ceil(), abs()
* typeOf(), asType()
* uuid()

Math Functions

* abs(n), floor(n), ceil(n)
* max(arr), min(arr), avg(arr), sum(arr), mod(x, y)

String Functions (dw::core::Strings)

* upper(s), lower(s), trim(s)
* contains(str, subStr)
* splitBy(str, regex)
* replace(str, old, new)
* startsWith(str, prefix)
* endsWith(str, suffix)
* joinBy(list, delimiter)

* length(str)

Date & Time Functions (dw::core::Dates)

* now(), today(), time()
* |duration| between two dates
* format(), parse()
* addDays(), subtractTime()

Array Functions (dw::core::Arrays)

* map(), mapObject()
* filter(), find(), every(), some()
* reduce(), pluck(), flatten()
* distinctBy(), groupBy()
* sortBy(), orderBy()

Object Functions (dw::core::Objects)

* mapObject(), pluck()
* filterObject(), mergeWith()
* keysOf(), valuesOf()
* containsKey(), renameKeys()
* orderBy()

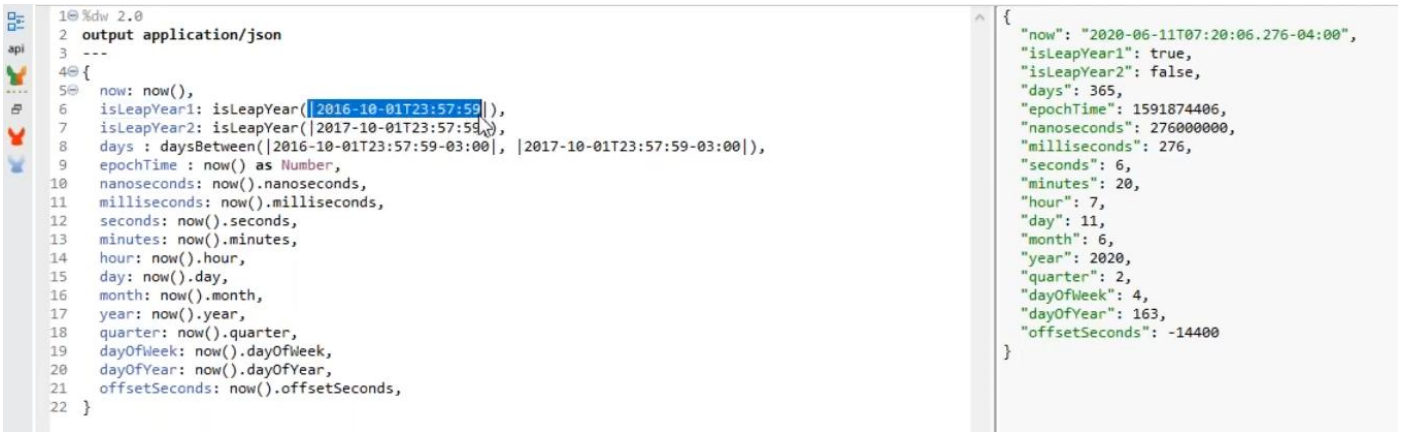
Type Conversion Functions

* as String, as Number, as Date, as Array
* as Object, as Boolean, as Null

Misc Utilities

* do { } (expression block)
* using (variable definition)
* when-otherwise (pattern matching)

DateTime :



```
%dw 2.0
output application/json
---
{
  now: now(),
  isLeapYear1: isLeapYear(|2016-10-01T23:57:59|),
  isLeapYear2: isLeapYear(|2017-10-01T23:57:59|),
  days : daysBetween(|2016-10-01T23:57:59-03:00|, |2017-10-01T23:57:59-03:00|),
  epochTime : now() as Number,
  nanoseconds: now().nanoseconds,
  milliseconds: now().milliseconds,
  seconds: now().seconds,
  minutes: now().minutes,
  hour: now().hour,
  day: now().day,
  month: now().month,
  year: now().year,
  quarter: now().quarter,
  dayOfWeek: now().dayOfWeek,
  dayOfYear: now().dayOfYear,
  offsetSeconds: now().offsetSeconds,
}
```

```
{
  "now": "2020-06-11T07:20:06.276-04:00",
  "isLeapYear1": true,
  "isLeapYear2": false,
  "days": 365,
  "epochTime": 1591874406,
  "nanoseconds": 276000000,
  "milliseconds": 276,
  "seconds": 6,
  "minutes": 20,
  "hour": 7,
  "day": 11,
  "month": 6,
  "year": 2020,
  "quarter": 2,
  "dayOfWeek": 4,
  "dayOfYear": 163,
  "offsetSeconds": -14400
}
```

Example:

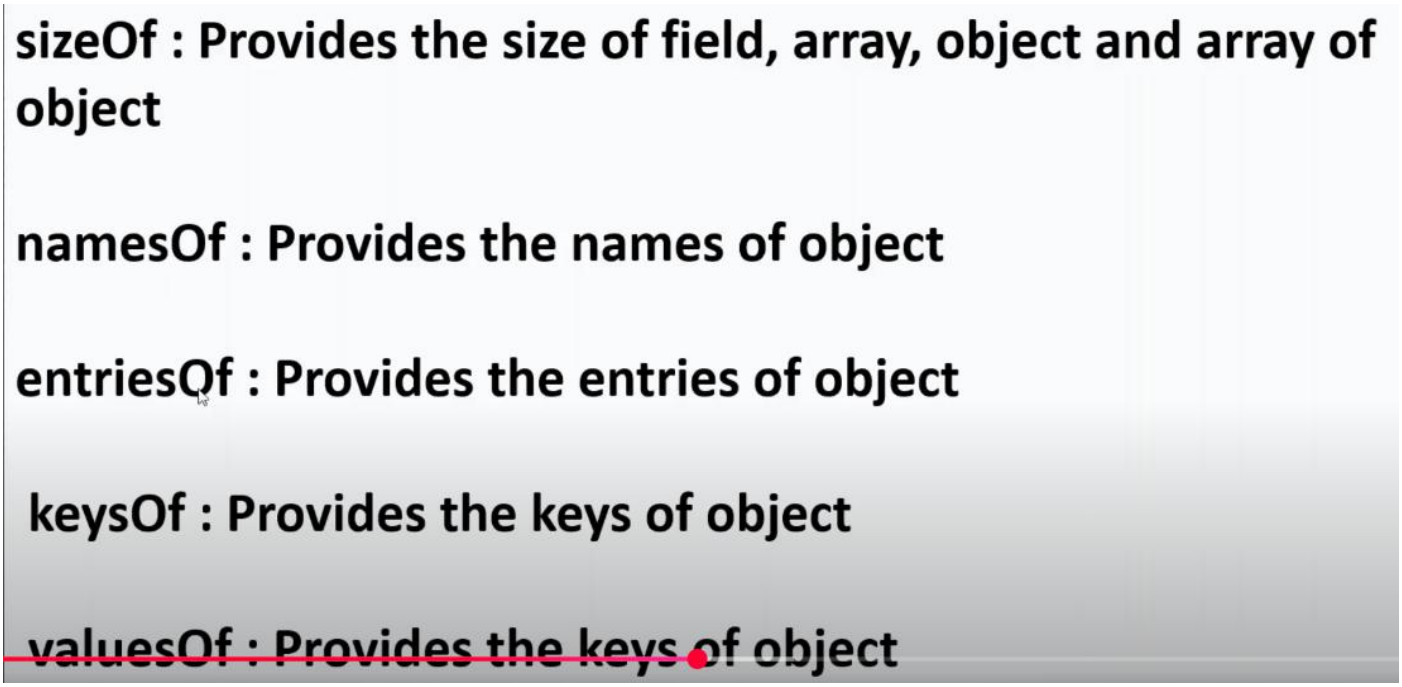
Calculate days of employee in company, if he is in more than 5 years or not.

```
%dw 2.0
output application/json
---
{
    "empID": payload[0].emp_id,
    "empName": payload[0].emp_name,
    "empJoinDate": payload[0].emp_join_date,
    "empExperience": daysBetween(payload[0].emp_join_date as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}, now() as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}) / 365,
    "gratuity": if((daysBetween(payload[0].emp_join_date as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}, now() as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}) / 365) > 5 ) "eligible" else "not-eligible"
}
```

(Or)

```
%dw 2.0
output application/json
var test = "I am test"
fun calExperience(joinDate) = daysBetween(payload[0].emp_join_date as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}, now() as LocalDateTime {format: "yyyy-MM-dd HH:mm:ss"}) / 365
---
{
    "empID": payload[0].emp_id,
    "empName": payload[0].emp_name,
    "empJoinDate": payload[0].emp_join_date,
    "empExperience": calExperience(payload[0].emp_join_date),
    "gratuity": if((calExperience(payload[0].emp_join_date)) > 5 ) "eligible" else "not-eligible"
}
```

==========



Example:

```
%dw 2.0
output application/json
var empName="Chinna"
var empSkills=["skill1", "skill2"] as Array
var employee = {
    "empID": "76868",
    "empName": "Chinna",
    "empStatus": "A"
}
var employeeList=[{
    "empID": "76868",
    "empName": "Chinna",
    "empStatus": "A"
},
{
    "empID": "76867",
    "empName": "John",
    "empStatus": "A"
},
{
    "empID": "76866",
    "empName": "Mark",
    "empStatus": "A"
},
{
    "empID": "76865",
    "empName": "Stephen",
    "empStatus": "A"
}]
---
{
    "sizeOf-FieldLevel": sizeOf(empName),
    "sizeOf-ArrayLevel": sizeOf(empSkills),
    "sizeOf-ObjectLevel": sizeOf(employee),
    "sizeOf-ArrayOfObjectLevel": sizeOf(employeeList),
    "namesOfEmployeeObject": namesOf(employee),
    "entriesOfEmployeeObject": entriesOf(employee),
    "keysOFEmployeeObeject": keysOf(employee),
    "valuesOfEmployeeObject": valuesOf(employee)
}
```

```
{
  "sizeOf-FieldLevel": 6,
  "sizeOf-ArrayLevel": 2,
  "sizeOf-ObjectLevel": 3,
  "sizeOf-ArrayOfObjectLevel": 4,
  "namesOfEmployeeObject": [
    "empID",
    "empName",
    "empStatus"
  ],
  "entriesOfEmployeeObject": [
    {
      "key": "empID",
      "value": "76868",
      "attributes": {

      }
    },
    {
      "key": "empName",
      "value": "Chinna",
      "attributes": {

      }
    },
    {
      "key": "empStatus",
      "value": "A",
      "attributes": {

      }
    }
  ],
  "keysOFEmployeeObeject": [
    "empID",
    "empName",
    "empStatus"
  ],
  "valuesOfEmployeeObject": [
    "76868",
    "Chinna",
    "A"
  ]
}
```

==========

map : To iterate over array of objects and results array of objects.

```
arrayOfObjectPayload map(item, indexOfItem) -> { }
```

mapObject : To iterate over a object and results object

```
objectPayload mapObject(value, key, index) -> {}
```

filter : To filter over array of objects

```
filter($.fieldname == value)
```

filterObject : To iterate over a object

```
filterObject($.fieldname == value)
```

Map Examples:

```
%dw 2.0
output application/json
var empList = [{
    empID: 100,
    empName: "Chinna",
    "empStatus": "A"
},
    {
    empID: 101,
    empName: "Mark",
    "empStatus": "I"
},
    {
    empID: 102,
    empName: "John",
    "empStatus": "I"
},
    {
    empID: 103,
    empName: "Stehphen",
    "empStatus": "A"
}]
---
empList map {
    ($$): $
}
```

```
[
  {
    "0": {
      "empID": 100,
      "empName": "Chinna",
      "empStatus": "A"
    }
  },
  {
    "1": {
      "empID": 101,
      "empName": "Mark",
      "empStatus": "I"
    }
  },
  {
    "2": {
      "empID": 102,
      "empName": "John",
      "empStatus": "I"
    }
  },
  {
    "3": {
      "empID": 103,
      "empName": "Stehphen",
      "empStatus": "A"
    }
  }
]
```

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName"": "Chinna",
    "empStatus": "A"
},
    {
    "empID"": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList map{
    "employeeID" : $.empID,
    "employeeName": $.empName,
    "empStatus": $.empStatus
}
```

```
[
  {
    "employeeID": 100,
    "employeeName": "Chinna",
    "empStatus": "A"
  },
  {
    "employeeID": 101,
    "employeeName": "Mark",
    "empStatus": "I"
  },
  {
    "employeeID": 102,
    "employeeName": "John",
    "empStatus": "I"
  },
  {
    "employeeID": 103,
    "employeeName": "Stehphen",
    "empStatus": "A"
  }
]
```

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList map(emp, indexOfEmp) -> {
    "employeeID" : emp.empID,
    "employeeName": emp.empName,
    "empStatus": emp.empStatus
}
```

```
[
  {
    "employeeID": 100,
    "employeeName": "Chinna",
    "empStatus": "A"
  },
  {
    "employeeID": 101,
    "employeeName": "Mark",
    "empStatus": "I"
  },
  {
    "employeeID": 102,
    "employeeName": "John",
    "empStatus": "I"
  },
  {
    "employeeID": 103,
    "employeeName": "Stehphen",
    "empStatus": "A"
  }
]
```

MapObject Examples:

Output Payload

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList[0] mapObject(empValue, empKey, indexOfEmp) -> {
    (empKey): indexOfEmp

}
```

```
{
  "empID": 0,
  "empName": 1,
  "empStatus": 2
}
```

Output Payload

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList[0] mapObject(empValue, empKey, indexOfEmp) -> {
    (empKey): empValue

}
```

```
{
  "empID": 100,
  "empName": "Chinna",
  "empStatus": "A"
}
```

Map and MapObjectexample:

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList map(emp, indexOfEmp) -> {
    (emp mapObject
        if ( ($) as String == "Chinna" ) {
        ($$): $,
        "empLocation": "India"
    } else {
        ($$): $
    })
}
```

```
[
  {
    "empID": 100,
    "empName": "Chinna",
    "empLocation": "India",
    "empStatus": "A"
  },
  {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
  },
  {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
  },
  {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
  }
]
```

Filter and map example:

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList filter($.empStatus == "A" and $.empName == "Chinna") map(emp, indexOfEmp) -> {
    "employeeID" : emp.empID,
    "employeeName": emp.empName,
    "empStatus": emp.empStatus
}
```

```
[
  {
    "employeeID": 100,
    "employeeName": "Chinna",
    "empStatus": "A"
  }
]
```

filterObject and mapObject example:

Output Payload

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
empList[0] filterObject(($) == 100) mapObject(empValue, empKey, indexOfEmp) -> {
    (empKey): empValue
}
```

```
{
  "empID": 100
}
```

==========

distinctBy : distinctBy(item, index) -> item

orderBy : orderBy(item, index) -> item

groupBy : groupBy(item, index) -> item

joinBy : array of elements joinBy “ ”

Examples:

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
    {
    "empID": 103,
    "empName": "John",
    "empStatus": "I"
},
    {
    "empID": 102,
    "empName": "Stehphen",
    "empStatus": "A"
}]
---
{
    "distinctByEmployees": empList distinctBy(item, index) -> item,
    "orderByEmployeesAsc": empList orderBy $.empID,
    "orderByEmployeesDesc": empList orderBy -$.empID,
    "groupByEmployeesByStatus": empList groupBy $.empStatus,
    "inactiveEmployees": (empList filter($.empStatus == "I")).empID joinBy ", ",
    "employees": empList orderBy $.empID distinctBy $.empID map((emp, indexOfEmp) -> {
        "employeeId": emp.empID,
        "employeeName": emp.empName,
        "employeeStatur": emp.empStatus
    })
}

```

==========

flatten : Flattens an array of arrays into a single, simple array.

Syntax: flatten(payload)

pluck: function use to get keys, values of a object

Flatten: it will combine 2 arrays into one.

```
%dw 2.0
output application/json
var empList = [{
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
},
    {
    "empID": 101,
    "empName": "Mark",
    "empStatus": "I"
},
[{
    "empID": 102,
    "empName": "Stephen",
    "empStatus": "I"
},
{
    "empID": 103,
    "empName": "John",
    "empStatus": "A"
}]
]
---
{
employees: flatten(empList)
}
```

```
{
  "employees": [
    {
      "empID": 100,
      "empName": "Chinna",
      "empStatus": "A"
    },
    {
      "empID": 101,
      "empName": "Mark",
      "empStatus": "I"
    },
    {
      "empID": 102,
      "empName": "Stephen",
      "empStatus": "I"
    },
    {
      "empID": 103,
      "empName": "John",
      "empStatus": "A"
    }
  ]
}
```

Pluck: it is used to get keys, values of a object.

Output Payload

```
%dw 2.0
output application/json
var employee = {
    "empID": 100,
    "empName": "Chinna",
    "empStatus": "A"
}
---
{
    "keysOfEmployee": employee pluck $$,
    "vaouesOfEmployee": employee pluck $
}
```

```
{
  "keysOfEmployee": [
    "empID",
    "empName",
    "empStatus"
  ],
  "vaouesOfEmployee": [
    100,
    "Chinna",
    "A"
  ]
}
```

Real example:

Output Payload

```
%dw 2.0
output application/json
---
{
    employeeID: payload.empID,
    employeeName: payload.empName,
    employeeAddress: payload.empAddress,
    employeeStatus: payload.empStatus,
    employeeJoinDate: payload.empJoinDate
}
```

```
{
  "employeeID": 898909,
  "employeeName": "Mark",
  "employeeAddress": {
    "houseNumber": "2-1-3987-1",
    "streetName": "Central Avenue",
    "city": "New York",
    "country": "USA"
  },
  "employeeStatus": null,
  "employeeJoinDate": "2020-06-08 08:43:12"
}
```

Here adrress is in Json format, we need it into single line.

```
%dw 2.0
output application/json
---
{
    employeeID: payload.empID,
    employeeName: payload.empName,
    employeeAddress: payload.empAddress pluck $ joinBy ", ",
    employeeStatus: payload.empStatus,
    employeeJoinDate: payload.empJoinDate
}
```

```
{
  "employeeID": 898909,
  "employeeName": "Mark",
  "employeeAddress": "2-1-8987-1, Central Avenue, New York, USA",
  "employeeStatus": null,
  "employeeJoinDate": "2020-06-08 08:43:12"
}
```

>> pluck use to put adress in a array
>> joinBy is used to put it in a single line
>> “ , “ is used to separate by comma.